Teoría de Números

La teoría de números es una rama de las matemáticas que estudia a los números enteros. Es un campo fascinante, porque muchos problemas que involucran números enteros son muy difíciles de resolver, aun cuando parecen simples a primera vista.

Como ejemplo, consideremos la siguiente ecuación:

$x^3+y^3+z^3=33$

Es fácil encontrar tres números reales x,y,z que satisfacen la ecuación.
Por ejemplo, podemos elegir:

$x=3, \quad y=\sqrt[3]{3}, \quad z=\sqrt[3]{3}.$

Sin embargo, en teoría de números sigue siendo un problema abierto determinar si existen tres enteros x,y,z que satisfagan esta ecuación [6].

Nos centraremos en conceptos básicos y algoritmos de la teoría de números. Asumiremos que todos los números son enteros, salvo que se indique lo contrario.

Primos y factores

Un número a se llama factor o divisor de un número b si a divide a b. Si a es un factor de b, escribimos a|b; en caso contrario, escribimos a∤b.

Por ejemplo, los factores de 24 son: 1,2,3,4,6,8,12 y 24. **Por tanto:** Un número a se llama **divisor o factor** de un número b si b es divisible por a, lo que significa que existe algún número entero k tal que b=ka.

Por convención, **1 y n** se consideran divisores de n. Un número n>1 es **primo** si sus únicos divisores(o factores positivos) **son 1 y n**. Los números mayores que 1 que no son primos se llaman **compuestos**.

Por ejemplo:

- 7, 19 y 41 son primos.
- 35 no es primo, porque 5·7=35.

Para todo número n>1, existe una **factorización prima única**. Todo número tiene una **factorización prima única**: una forma de descomponerlo como producto de primos, de la siguiente manera: $n=p_1^{a1}\,p_2^{a2}\ldots p_k^{ak}$

donde cada pi es un número primo distinto y cada **ai** es un número entero positivo que **indica cuántas veces se repite ese primo**.

Ejemplos: $84=\mathbf{2^2\times3^1\times7^1}$

Fórmula de la progresión geométrica

¿Quienes son los número de factores de un número?: Los números de factores de un número **son todos los divisores positivos que lo dividen exactamente**, y su cantidad **indica cuántos números distintos pueden multiplicarse** para formar dicho número.

Ejemplo: Si tienes un número n descompuesto en **factorización prima**:

$n = p_1^{a1}\, p_2^{a2} \ldots p_k^{ak}$

- donde cada pi(p1,p2,…,pk): Son números primos distintos.
- cada **ai(α1,α2,…,αk):** Son los **exponentes** correspondientes e **indica cuántas veces se repite ese primo**.

Entonces **el número de factores de un número n** se calcula así:

$\sigma(n) = (a_1 + 1\ ) * (a_2 + 1\ ) * (a_3 + 1\ ) * \ldots . (a_k + 1\ )$

¿Por qué se usa αi+1?

Cada primo pi puede aparecer en un factor **0 veces, 1 vez, 2 veces, ..., hasta αi veces**.

Si pi^2 aparece en la factorización, entonces en los factores posibles puedes usarlo de **3** formas (usar $2^0, 2^1, 2^2$) es por eso que $\alpha_2 + 1 = 2 + 1 = 3$

Generalizando la formula quedaría que el número de factores de un número n es

$\sigma(n) = \prod_{i=1}^{k} (ai + 1\ )$ porque para cada primo pi, hay **αi+1** formas de elegir cuántas veces aparece en el factor. Por ejemplo, el número de factores de 84 es τ(84)=3·2·2=12. Esto se debe a que $84 = 2^2 \times 3^1 \times 7^1$ **se multiplican los exponentes (2+1)*(1+1)*(1+1)= 3·2·2=12**

Los factores son 1,2,3,4,6,7,12,14,21,28,42 y 84. Con eso comprobamos que son 12 factores

La **suma de los factores de n** es: 1+2+3+4+6+7+12+14+21+28+42+84 =224

$$\sigma(n) = \prod_{i=1}^{k} (1 + pi + \cdots + pi^{ai}\ ) = \prod_{i=1}^{k} \frac{(pi^{ai+1} - 1\ )}{pi - 1}$$

donde la última fórmula se basa en la **fórmula de la progresión geométrica**. Por ejemplo, la suma de factores de 84 es:

$$\sigma(84) = \frac{2^3 - 1}{2 - 1} * \frac{3^2 - 1}{3 - 1} * \frac{7^2 - 1}{7 - 1} = 7 \cdot 4 \cdot 8 = 224$$

El **producto de los factores de n** es

Sea un número n con τ(n) factores (es decir, **τ(n)** es el **número total de divisores** de n).

El **producto de todos los factores** de n se denota como μ(n) y se calcula con la fórmula:

$\mu(n) = n^{\tau(n)/2}$

¿Por qué se usa esta fórmula?

Piensa en todos los factores de n. Cada factor d **se puede emparejar con otro factor** n/d para que el **producto del par sea** n.

Por ejemplo, si n=84:

- Los factores de 84 son: 1,2,3,4,6,7,12,14,21,28,42,84
- Podemos emparejarlos así: $1\cdot84,\ 2\cdot42,\ 3\cdot28,\ 4\cdot21,\ 6\cdot14,\ 7\cdot12$

Cada par tiene **producto = 84**, y hay τ(84)/2=12/2=6 pares.

Por tanto podemos formar **τ(n)/2** pares a partir de los factores, cada uno con producto n. Por ejemplo, los factores de 84 producen los pares 1·84, 2·42, 3·28, etc., y el producto de los factores es $\mu(84)=84^6=351298031616$.

Producto total

Si cada uno de esos pares da 84, y hay 6 pares, entonces el **producto de todos los factores** es:

$\mu(84)=84^6=351298031616$

Resumen

- Emparejamos los factores de manera simétrica: el más pequeño con el más grande, el siguiente más pequeño con el siguiente más grande, etc.
- Cada par da n como producto.
- Como hay τ(n)/2 pares, multiplicamos n tantas veces: $\mu(n)=n^{\tau(n)/2}$
- de manera indirecta se puede relacionar con un razonamiento similar al que usó **Gauss** para sumar series, pero no es exactamente lo mismo

Número perfecto

Un número n se llama **número perfecto** si $n=\sigma(n)-n$, es decir, n es igual a la suma de sus factores entre 1 y n−1. Por ejemplo, 28 es un número perfecto, porque

$28=1+2+4+7+14$.

Número de primos

Es fácil demostrar que **existe un número infinito de primos**. Si el número de primos fuera finito, podríamos construir un conjunto: $P=\{p1,p2,\ldots,pn\}$ que contuviera todos los primos. Por ejemplo, p1=2, p2=3, p3=5, y así sucesivamente. Sin embargo, usando P, podríamos formar un nuevo primo

$p1p2\ldots pn+1$

que es mayor que todos los elementos de P. Esto es una contradicción, y por lo tanto el número de primos **tiene que ser infinito**.

Densidad de primos

La densidad de primos indica con qué frecuencia aparecen los primos entre los números. Sea π(n) el número de primos entre 1 y n. Por ejemplo, **π(10)=4**, porque hay 4 primos entre 1 y 10: 2, 3, 5 y 7.

Es posible demostrar que

$\pi(n) \approx \frac{n}{\ln n}$, lo que significa que los primos son bastante frecuentes. Por ejemplo, el número de primos entre 1 y 10^6 es $\pi(10^6)=78498$, y $10^6/\ln 10^6 \approx$ **72382**.

Tener en cuenta que:

- **$\ln(10) \approx 2.302585$**
- Usando la propiedad **$\ln(a^b)=b \cdot \ln(a)$**:
  - $\ln(10^6)=6 \cdot \ln 10 \approx 6 \cdot 2.302585 \approx$ **13.81551**
  - $\ln 10 \approx 2.3026$, $\ln 1{,}000{,}000 \approx 13.8155$
  - por tanto $1{,}000{,}000 / \ln 1{,}000{,}000 \approx 13.8155 \approx$ **72382,5**

No es **exacto la densidad de primo**, es una **aproximación** muy buena. Pero sirve para **entender la densidad y la frecuencia de los primos** entre los números. Para mejorar la aproximación, se usa el **logaritmo integral y este último no debe ir a concurso, porque se da en universidad o quizas algunos pre, lo dan en 12 grado**

$$\int_2^n \frac{dt}{\ln t}$$

Conjeturas

Existen muchas conjeturas que involucran primos. La mayoría de las personas piensa que las conjeturas son verdaderas, pero nadie ha podido demostrarlas. Por ejemplo, las siguientes conjeturas son famosas:

- **Conjetura de Goldbach**: Cada número entero par n>2 puede representarse como una **suma n=a+b** de manera que tanto a como b sean primos.

  Ejemplo: Donde n=28

  - 28=11+17 (ambos son primos)
  - 28=5+23 (ambos son primos)

  Uso en concursos:

  - Se puede usar en ejercicios que pidan **encontrar pares de primos que sumen un número par dado**.

- **Conjetura de los primos gemelos**: Existe un **número infinito de pares** de la forma {p,p+2}, donde tanto p como p+2 son primos.

  Ejemplo:

  - p=11 → {11,13}
  - p=17 → {17,19}

  Uso en concursos:

  - Se puede usar en **problemas de identificación de pares de primos con diferencia 2**.

- **Conjetura de Legendre**: **Siempre hay un primo entre los números n^2 y $(n+1)^2$,** donde n es cualquier número entero positivo.

  Ejemplo:

  - **n=4** → intervalos: 16 a 25
    - Primos en ese intervalo: 17,19,23
  - **n=5** → intervalo: 25 a 36
    - Primos: 29,31

  Uso en concursos:

  - Se puede usar para **problemas de existencia de primos en intervalos cuadrados**.

NOTA: Uso en concursos:

- No se pueden usar para demostrar la conjetura completa, porque aún no está demostrada para todos los n

Algoritmos básicos

Si un número n no es primo, puede representarse como un producto a*b, donde $a \leq \sqrt{n}$ o $b \leq \sqrt{n}$, por lo que ciertamente tiene un factor entre 2 y $\lfloor \sqrt{n} \rfloor$. Usando esta observación, podemos tanto **probar si un número es primo** como **encontrar la factorización prima de un número** en tiempo $O(\sqrt{n})$.

La siguiente **función** `prime` verifica si el número dado n es primo. La función intenta dividir n entre todos los números entre 2 y $\lfloor \sqrt{n} \rfloor$, y si ninguno de ellos divide a n, entonces n es primo.

```
// Función que devuelve true si n es primo, false si no lo es
bool prime(int n) {
  if (n < 2) return false;  // Los números menores que 2 no son primos
  for (int x = 2; x*x <= n; x++) { // Recorre posibles divisores hasta ⌊√n⌋
    if (n % x == 0) return false; // Si n es divisible por x, no es primo
  }
```

```
    return true; // Si no se encontró ningún divisor, n es primo
}
```

Depurando código arriba N=17

Iteración	`x`	`x*x`	Condición `x*x <= 17`	`17 % x`	¿Es divisor?
1	2	4	4 ≤ 17 → ✅	17 % 2 = 1	❌
2	3	9	9 ≤ 17 → ✅	17 % 3 = 2	❌
3	4	16	16 ≤ 17 → ✅	17 % 4 = 1	❌
4	5	25	25 ≤ 17 → ❌ → se detiene	–	–

👉 Como no encontró ningún divisor, la función retorna **true**: `17` es primo.

La siguiente **función `factors`** construye un vector que contiene la factorización prima de n. La función divide n por sus factores primos y los añade al vector. El proceso termina cuando el número restante n no tiene factores entre 2 y $\lfloor\sqrt{n}\rfloor$. Si n>1, entonces n es primo y es el último factor.

```
// Función que devuelve un vector con los factores primos de n
vector<int> factors(int n) {
    vector<int> f; // Vector para almacenar los factores primos
    for (int x = 2; x*x <= n; x++) {  // Recorre posibles divisores hasta ⌊√n⌋
        while (n % x == 0) { // Mientras x divida a n →  entonces va a repetir el ciclo
            f.push_back(x); // Agrega x al vector de factores
            n /= x;  // Divide n entre x para seguir factorizando  n = n / x
        }
    }
    if (n > 1) f.push_back(n); // Si queda un número mayor que 1, también es primo
    return f; // Devuelve el vector con todos los factores primos
}
```

Depurando código arriba N=60

Iteración externa	`x`	`x*x <= n`	`n % x`	Acción	`n` (nuevo)	`f` (factores encontrados)
1	2	2*2=4 ≤ 60 ✅	60 % 2 = 0 ✅	`f.push_back(2)` → `n/=2`	30	[2]
1 (otra vez)	2	2*2=4 ≤ 30 ✅	30 % 2 = 0 ✅	`f.push_back(2)` → `n/=2`	15	[2, 2]
1 (otra vez)	2	2*2=4 ≤ 15 ✅	15 % 2 = 1 ❌	Sale del `while`	15	[2, 2]
2	3	3*3=9 ≤ 15 ✅	15 % 3 = 0 ✅	`f.push_back(3)` → `n/=3`	5	[2, 2, 3]
2 (otra vez)	3	3*3=9 ≤ 5 ❌	–	Termina `while`	5	[2, 2, 3]
3	4	4*4=16 ≤ 5 ❌	–	No entra al `for`	5	[2, 2, 3]
Final	–	–	–	Como `n=5 > 1`, hace `f.push_back(5)`	–	[2, 2, 3, 5]

Resultado: `factors(60)` devuelve **[2, 2, 3, 5]**. que corresponde a la factorización prima 60=22×3×5.

Nota que cada factor primo aparece en el vector tantas veces como divide al número. Por ejemplo, $24=2^3 \cdot 3$, así que el resultado de la función es [2,2,2,3].

Lo mismo explicado de otra manera como algoritmo

Algoritmo: Factorización prima de un número

Algorithm: Finds the prime factorization of a number

```
Function factor
    Input  : n, the number to be factorized
    Output : v, a list of all the prime factors
    v ← empty list
    for i ← 2 to ⌊√n⌋ do
        while n is divisible by i do
            n ← n/i
            Add i to the list v
        end
    end
    return v;
```

Función `factor`
Entrada: **n**, el número que queremos factorizar

Salida: **v**, una lista con todos los factores primos de n

---

1. Crear una lista vacía v.
2. Para cada número i desde 2 hasta $\sqrt{n}$:
    - Mientras n sea divisible entre i:
        - Dividir n entre i.
        - Agregar i a la lista v.
3. Si al final n>1, agregar n a la lista (porque n es primo).
4. Devolver la lista v.

Ejemplo paso a paso

Factorizar n=60:

- i=2:
    - 60 ÷ 2 = 30 → agregar 2
    - 30 ÷ 2 = 15 → agregar 2
- i=3:
    - 15 ÷ 3 = 5 → agregar 3
- i=5:

- $5 \div 5 = 1 \rightarrow$ agregar 5

Lista final: v=[2,2,3,5]

Factorización prima: $60=2^2\times3\times5$

Este algoritmo se ejecuta en **tiempo O(n)**, porque el bucle `for` revisa divisibilidad como máximo para $\sqrt{n}$ valores. Aunque dentro del `for` hay un bucle `while`, al dividir n entre i varias veces, el **valor de n se va reduciendo rápidamente**. Esto significa que el bucle externo (`for`) termina recorriendo menos iteraciones de las que parecería en un inicio, lo cual en la práctica **hace que el algoritmo sea más rápido**.

Veamos un ejemplo de cómo funciona este algoritmo, para n=252.

i	n	v
2	252	$\{\}$
2	126	$\{2\}$
2	63	$\{2, 2\}$
3	21	$\{2, 2, 3\}$
3	7	$\{2, 2, 3, 3\}$

En este punto, el bucle **for** termina, porque i ya es 3, lo cual es mayor que $|\sqrt{7}|$. En el último paso, agregamos el 7 a la lista de factores v, porque de otro modo no se añadiría, obteniendo finalmente la factorización prima: **{2, 2, 3, 3, 7}**.

Mi código en java:

```
import java.util.*;

public class FactorizacionPrima {

    // Función que devuelve la lista de factores primos
    public static List<Integer> factorizar(int n) {
        List<Integer> factores = new ArrayList<>();

        // Recorremos posibles divisores hasta raíz de n
        for (int i = 2; i * i <= n; i++) {
            while (n % i == 0) {
                factores.add(i);
                n /= i; // Reducimos n
            }
        }

        // Si al final n > 1, es primo y debe agregarse
        if (n > 1) {
            factores.add(n);
        }

        return factores;
    }
```

```
    public static void main(String[] args) {
        int n = 252;
        List<Integer> resultado = factorizar(n);

        System.out.println("Factorización prima de " + n + ": " + resultado);
        // También mostramos en forma de potencias
        Map<Integer, Integer> potencias = new LinkedHashMap<>();
        for (int f : resultado) {
            potencias.put(f, potencias.getOrDefault(f, 0) + 1);
        }

        System.out.print("Forma de producto: ");
        boolean primero = true;
        for (Map.Entry<Integer, Integer> e : potencias.entrySet()) {
            if (!primero) System.out.print(" × ");
            System.out.print(e.getKey() + "^" + e.getValue());
            primero = false;
        }
        System.out.println();
    }
}
Factorización prima de 252: [2, 2, 3, 3, 7]
Forma de producto: 2^2 × 3^2 × 7^1
```

Criba de Eratóstenes

La criba de Eratóstenes es un algoritmo de preprocesamiento que construye un arreglo mediante el cual podemos **verificar eficientemente** si un número dado entre 2…n es primo y, si no lo es, encontrar uno de sus factores primos.

El algoritmo construye un **arreglo `sieve`** cuyas posiciones 2,3,…,n se utilizan.

- El valor **`sieve[k] = 0`** significa que k es primo.
- El valor **`sieve[k] <> 0`** significa que k no es primo y uno de sus factores primos es `sieve[k]`.

El algoritmo itera por los números 2…n uno por uno. **Siempre que se encuentra un nuevo primo x**, el algoritmo registra que los múltiplos de x (2x,3x,4x,…) **no son primos**, porque x los divide.

Por ejemplo, si **n=20**, el arreglo es el siguiente:

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	0	2	0	3	0	2	3	5	0	3	0	7	5	2	0	3	0	5

El siguiente código implementa la criba de Eratóstenes. El código asume que cada elemento de `sieve` es inicialmente cero:

```
// Recorre todos los números desde 2 hasta n
for (int x = 2; x <= n; x++) {
   if (sieve[x]) continue;
   // Si sieve[x] ya tiene un valor distinto de 0, significa que 'x' no es primo,
   // entonces se salta (ya fue marcado antes como múltiplo de otro número).

   // Si sieve[x] == 0, significa que 'x' es primo.
   // Ahora marcamos todos los múltiplos de 'x' en la criba.
   for (int u = 2*x; u <= n; u += x) {   // u = u+x
      sieve[u] = x;
      // Guardamos en sieve[u] el divisor 'x' (un factor primo de u).
   }
}
```

Depurando codigo con **`n = 10`**

Paso	x	¿sieve[x] es 0?	Acción	Valores actualizados en sieve
1	2	sieve[2]=0 → primo	Marca múltiplos de 2	sieve[4]=2, sieve[6]=2, sieve[8]=2, sieve[10]=2
2	3	sieve[3]=0 → primo	Marca múltiplos de 3	sieve[6]=3 (ya era 2, ahora sobreescrito), sieve[9]=3
3	4	sieve[4]=2 ≠ 0 → no primo	**continue**	–
4	5	sieve[5]=0 → primo	Marca múltiplos de 5	sieve[10]=5 (sobreescrito, antes era 2)
5	6	sieve[6]=3 ≠ 0 → no primo	**continue**	–
6	7	sieve[7]=0 → primo	Marca múltiplos de 7	Ninguno ≤ 10
7	8	sieve[8]=2 ≠ 0 → no primo	**continue**	–
8	9	sieve[9]=3 ≠ 0 → no primo	**continue**	–
9	10	sieve[10]=5 ≠ 0 → no primo	**continue**	–

El bucle interno del algoritmo se ejecuta n/x veces para cada valor de x. Por lo tanto, una cota superior para el tiempo de ejecución del algoritmo es la **suma armónica**:

$$\sum_{x=2}^{n} n/x = n/2 + n/3 + n/4 \cdots + n/n = O(n \log n)$$

De hecho, el algoritmo es más eficiente, porque el bucle interno se ejecutará solo si el número x es primo. Se puede demostrar que el tiempo de ejecución del algoritmo es **solo O(n log log n)**, una complejidad muy cercana a O(n).

Algoritmo de Euclides

El **máximo común divisor** de los números a y b, **gcd**(a,b), es el mayor número que divide a ambos a y b. El **mínimo común múltiplo** de a y b, **lcm**(a,b), es el número más pequeño divisible entre ambos a y b.

Por ejemplo: gcd(24,36)=12 y lcm(24,36)=72

El máximo común divisor y el mínimo común múltiplo están conectados de la siguiente manera:

$$lcm(a,b) = \frac{ab}{gcd(a,b)}$$

El **mínimo común múltiplo (lcm)** está relacionado: gcd(**a**,**b**)*lcm(**a**,**b**)= **a*b**

Ejemplo:

gcd(**24**,**36**)=**12** y lcm(**24**,**36**)=**72**

12·**72**=**24**·**36**

Algoritmo de Euclides : El algoritmo de Euclides proporciona una manera eficiente de encontrar el máximo común divisor(**GCD**) de dos números. El máximo común divisor (MCD) de dos enteros a y b es el entero más grande que es un factor de ambos a y b. Para encontrar el MCD de dos números, usamos el Algoritmo de Euclides, que es el siguiente:

$$\gcd(a,b) = \begin{cases} a & b = 0 \\ \gcd(b, a \bmod b) & b \neq 0 \end{cases}$$

Encontrar el **MCD** de dos números puede hacerse en tiempo O(log n), donde n = min(a, b). El **mínimo común múltiplo (MCM)** de dos enteros a y b es el entero más pequeño divisible por ambos a y b. El MCM puede calcularse fácilmente a partir de la siguiente propiedad con el MCD:

$$lcm(a,b) = \frac{ab}{gcd(a,b)}$$

Si queremos calcular el MCD o MCM de más de dos elementos, podemos hacerlo de dos en dos, en cualquier orden. Por ejemplo: gcd(**a1**,**a2**, **a3, a4**)= **gcd(a1, gcd(a2, gcd(a3, a4)))**

Idea del Algoritmo de Euclides, Se basa en esta propiedad:

gcd(**a**,**b**)=gcd(**b**, **a** mod **b**)

Es decir:

- El gcd de dos números **no cambia si reemplazamos el mayor por su resto** al **dividirlo** entre el **menor**.
- Si uno de los números se hace 0, el otro es el gcd.

Ejemplo: gcd(24,36)

- **gcd(24,36)**
  - 36÷24=1 con resto **12** → gcd(36,24)=gcd(24,12)
  - 24÷12=2 con resto **0** → gcd(24,12)=gcd(12,0)
  - Resultado: **12**

1. gcd(24,36)=gcd(36,24)
2. gcd(36,24)=gcd(24,12)
3. gcd(24,12)=gcd(12,0)
4. Resultado: **12**

Por tanto: gcd(24,36) = gcd(36,24) = gcd(24,12) = gcd(12,0)= **12**

Este algoritmo es muy fácil de implementar usando una **función recursiva**, de la siguiente manera:

```
int gcd(int a, int b) {
    if (b == 0) return a; // Caso base: si b es 0, el MCD es a
     return gcd(b, a % b); // Llamada recursiva: ahora calculamos gcd(b, resto de a/b)
}
```

Depuración con ejemplo: `gcd(48, 18)`

Llamada recursiva	`a`	`b`	`a % b`	Acción
1	48	18	48 % 18 = 12	No entra al `if`, llama `gcd(18, 12)`
2	18	12	18 % 12 = 6	No entra al `if`, llama `gcd(12, 6)`
3	12	6	12 % 6 = 0	No entra al `if`, llama `gcd(6, 0)`
4	6	0	–	`b == 0` → retorna 6

Se puede demostrar que el algoritmo de Euclides funciona en **tiempo O(log n)**, donde n=min(a,b). El peor caso para el algoritmo ocurre cuando a y b son **números consecutivos de Fibonacc**i.

- gcd(13,8)
  - Dividimos 13 entre 8: 13=1·8+5
  - Resto = **5** → gcd(13,8)=gcd(8,5)
- gcd(8,5)
  - 8=1·5+3

    - Resto = **3** → gcd(8,5)=gcd(5,3)
- gcd(5,3)
    - 5=1·3+2
    - Resto = **2** → gcd(5,3)=gcd(3,2)
- gcd(3,2)
    - 3=1·2+1
    - Resto = **1** → gcd(3,2)=gcd(2,1)
- gcd(2,1)
    - 2=2·1+0
    - Resto = **0** → gcd(2,1)=gcd(1,0)=1

Por ejemplo: gcd(13,8)=gcd(8,5)=gcd(5,3)=gcd(3,2)=gcd(2,1)=gcd(1,0)=1

Resultado final: **gcd(13, 8) = 1**

Función totiente de Euler

Dos números a y b son **coprimos** si gcd(a,b)=1. La función totiente de Euler φ(n) **da la cantidad de números coprimos** con n **entre 1 y n**.

Por ejemplo: **φ(12)=4**, porque **1, 5, 7 y 11** son coprimos con 12.

El valor de φ(n) puede calcularse a partir de la factorización prima de n usando la fórmula:

$$\varphi(n)=\prod_{i=1}^{k} pi^{ai-1}\,(pi-1)$$

Cálculo usando factorización prima

Si factorizamos n en primos: $n=p_1^{a1}\,p_2^{a2}\ldots p_k^{ak}$

La fórmula de Euler es: $\varphi(n)=n*(1-\frac{1}{p_1})*(1-\frac{1}{p_2})*(1-\frac{1}{p_k})$

$$\varphi(12)=n*(1-\frac{1}{2})*(1-\frac{1}{3})=(12*\frac{1}{2}*\frac{2}{3})=4$$

$\varphi(12)=2^1\cdot(2-1)\cdot3^0\cdot(3-1)=4$

Caso especial: n primo

Si n es primo, todos los números de 1 a n−1 son coprimos con n:

$$\phi(n)=n-1$$

Ejemplo: n=7 → ϕ(7)=6 porque 1, 2, 3, 4, 5, 6 son coprimos con 7.